Machine Learning · 2019-20

CSE 475 | Session 2019-20 | June 2024 final

Model study answers. Every listed option is addressed. Question titles are paraphrased; exact wording and original diagrams are in the source pages appended to this pack.

Printed header: June 2024, session 2019-20; 3 hours, 100 marks. Every listed option is covered. The paper's wording about logistic regression and SVM is clarified rather than repeated as a false claim.

Quick revision

Standardization: z = (x - mean) / standard deviation. Fit statistics on training data only.

Gini(D) = 1 - sum(p_k squared). Split impurity = sum((n_j/n) Gini(D_j)).

Perceptron: z = w.x + b; step(z) = 1 for z >= 0, otherwise 0.

Precision = TP/(TP+FP); recall = TP/(TP+FN); accuracy = (TP+TN)/N.

F1 = 2TP/(2TP+FP+FN). Naive Bayes score(c) = P(c) product P(x_j|c).

Iterations/epoch = ceiling(N/batch size), when retaining the last partial batch.

Entropy H = −sum(p log2 p); information gain = parent entropy − weighted child entropy.

Valid convolution size = floor((input − kernel)/stride)+1 per axis.

1(a) | Plurality voting

Each classifier or neighbor votes for a class; choose the class with the most votes. It does not require more than half of all votes. Example: A=4, B=3, C=2; A wins with 4 of 9 votes. Define a tie rule in advance.

Revision answer: Choose the class receiving the largest number of votes.

1(b) | Two loss functions

Mean squared error: L=(1/n)sum(y−prediction)^2, commonly used for regression. Binary cross-entropy: L=−(1/n)sum[y log(p)+(1−y)log(1−p)], used for binary classification probabilities. Loss measures prediction error to guide fitting.

Revision answer: MSE and binary cross-entropy.

1(c) | Hyperparameters and examples

Hyperparameters are settings chosen outside the main parameter-fitting process. Examples: k in k-NN, tree maximum depth, neural-network learning rate, regularization strength and batch size. Tune them on validation data rather than the final test set.

Revision answer: Hyperparameters control the learning setup; weights are learned parameters.

1(d) | Bootstrap sample

Draw observations from the original dataset with replacement. A sample often has the same number of rows as the original, but repeated rows occur and some originals are absent. Example: from [A,B,C,D], one bootstrap sample is [B,B,D,A]. This is used in bagging.

Revision answer: A random sample drawn with replacement.

1(e) | Two reinforcement-learning examples

A robot learns navigation actions from rewards for reaching a goal and penalties for collisions. A game-playing agent learns moves from rewards for winning or increasing score. In both, actions affect future states and the objective is cumulative reward, rather than simply copying independent labeled examples.

Revision answer: Robot navigation and game-playing through interaction and rewards.

1(f) | When decision-tree entropy is zero

H=−sum p_k log2(p_k). If every observation belongs to one class, its probability is 1 and the others are 0, so H=−1 log2(1)=0, using the convention 0 log2(0)=0. This is a pure node.

Revision answer: Entropy is zero when the node contains only one class.

1(g) | High bias

High bias is systematic error from overly restrictive assumptions. A model too simple for the true pattern may underfit, performing poorly on both training and validation data. Example: fitting a straight line to a strongly curved relationship. Increase suitable model capacity, improve features or reduce excessive regularization.

Revision answer: An overly restrictive model systematically underfits the pattern.

2(a) | Explain model.compile and model.fit

compile configures learning: optimizer="rmsprop" chooses the RMSprop update method; loss="binary_crossentropy" compares binary labels with output probabilities; metrics=["accuracy"] requests reported classification accuracy. compile itself does not train the weights.

fit(x_train,y_train,epochs=4,batch_size=512) trains on the examples and labels for four passes through the training set, normally updating after each mini-batch of up to 512 examples. With N examples and the last partial batch retained, each epoch has ceiling(N/512) updates. N is not supplied, so a numerical count cannot be determined. The model needs a suitable binary output; accuracy is a reporting metric, not the loss optimized here.

Reading: Keras training API - https://keras.io/api/models/model_training_apis/

Revision answer: Configure RMSprop, binary cross-entropy and accuracy; train 4 epochs with batch size 512.

2(b) | Perceptron limitation and logistic regression

A single perceptron learns a linear separating boundary and is guaranteed to converge only when data are linearly separable under the usual assumptions. With overlapping classes it can keep changing weights without finding a perfect separator; its hard step output is not a calibrated probability.

Logistic regression uses $p(y=1|x)=\text{sigmoid}(w\cdot x+b)$, a smooth cross-entropy objective and probabilistic outputs. It can fit a best-compromise linear classifier to overlapping data with regularization. Important correction: ordinary logistic regression still has a linear decision boundary in its input features; it does not by itself solve XOR or arbitrary nonlinear separation. Use nonlinear features, kernels or multilayer models when needed.

Revision answer: Logistic regression gives smooth probabilistic fitting; both ordinary models have linear boundaries.

2(c) | Dimensionality reduction and PCA compression

Reducing dimensions can reduce storage/computation, remove redundant directions, improve visualization and sometimes reduce noise/overfitting. It may also discard predictive information.

PCA steps: split data; fit training means and scale features if their units require it; center the training matrix X; obtain principal directions from its covariance eigendecomposition or SVD; sort by decreasing variance; retain k directions. Compressed representation Z=X_centered W_k. Reconstruction X_hat=Z W_k^T+mean. When k<original dimension, reconstruction loses variation in discarded directions. Choose k using explained variance and task validation, and reuse the training transformation on test data. PCA maximizes retained variance, not class separation.

Revision answer: PCA projects centered data onto k high-variance orthogonal directions.

2(d) | TF-IDF and word relevance

Term frequency measures how often a term occurs in a document, often normalized. Inverse document frequency downweights terms appearing in many documents: a simple form is idf(t)=log(N/df(t)); practical variants use smoothing. TF-IDF = tf(t,d) x idf(t). A frequent word in one document that is rare across the corpus receives a large weight. A ubiquitous word receives a small weight. Example: in a corpus of 100 documents, a term in only 2 documents has idf=log(50).

Revision answer: High local frequency plus low corpus frequency gives high TF-IDF.

2(e) | Find the most discriminative feature

A discriminative feature helps distinguish target classes. On training data, compare each feature's association with the class using an appropriate measure: information gain for categorical splits, or a justified statistical score for numerical inputs. Validate usefulness using a model and held-out or cross-validated permutation importance.

For information gain: compute parent class entropy, partition by a feature, calculate weighted child entropy and subtract it. Larger gain means greater impurity reduction for that split. Use identical evaluation conditions for all candidates and avoid test-set leakage. Correlated features and high-cardinality splits can mislead simple rankings; no dataset is supplied here, so no particular feature can be named.

Revision answer: Compare label-separation scores and validate predictive contribution without test leakage.

2(f) | SVM slack variables and nonlinear separation

For labels $y_i \in \{-1,+1\}$, soft-margin SVM minimizes $(1/2)||w||^2+C \sum \xi_i$ subject to $y_i(w \cdot \varphi(x_i)+b) \geq 1-\xi_i$ and $\xi_i \geq 0$. $\xi=0$ meets the margin; $0<\xi<1$ is inside it but correctly classified; $\xi>1$ is on the wrong side ($\xi=1$ lies at the decision boundary).

Slack permits overlap or noisy points at a penalty; large C penalizes violations more strongly. Slack alone does not create a nonlinear boundary. A nonlinear feature map φ, typically computed through a kernel, supplies nonlinear separation in the original input space.

Revision answer: Slack permits penalized margin violations; kernels or nonlinear features handle nonlinear boundaries.

3(a) | Spam confusion matrix and all metrics

Take Spam as positive. True positives: rows 2,4,8 → TP=3. False negatives: rows 1,9 → FN=2. False positive: row 5 → FP=1. True negatives: rows 3,6,7,10 → TN=4.

Matrix with actual rows and predicted columns [Spam, Not Spam]: actual Spam [3,2]; actual Not Spam [1,4]. Total=10. Accuracy=(TP+TN)/10=(3+4)/10=0.70=70%. Sensitivity/recall=TP/(TP+FN)=3/5=0.60=60%. Precision=TP/(TP+FP)=3/4=0.75=75%. F1=2TP/(2TP+FP+FN)=6/9=0.6667≈66.67%.

Revision answer: TP=3, FN=2, FP=1, TN=4; accuracy 70%, recall 60%, precision 75%, F1 66.67%.

3(b)(i) | Feature scaling and standardization

Distance-based models can let a numerically large feature dominate; gradient optimization can also be poorly conditioned. Standardization uses z=(x-mean)/standard deviation. Example: mean height 160 cm, standard deviation 10 cm, height 175 cm gives z=(175-160)/10=1.5. Calculate mean and standard deviation from training data, then reuse them for validation/test. Trees generally do not require scaling for their split decisions.

Revision answer: Standardization expresses features in standard-deviation units.

3(b)(ii) | How L1 regularization selects features

Minimize prediction loss + lambda sum|w_j|. The absolute-value penalty favors sparse solutions, pushing some coefficients exactly to zero. Features whose coefficients become zero are effectively excluded. Increasing lambda generally increases shrinkage; choose it with validation and scale features when coefficient magnitudes must be comparable.

Revision answer: L1 makes some weights zero, creating a sparse feature set.

3(c)(i) | Steps for training a neural network

Prepare and split the data; scale inputs where appropriate. Choose architecture, activations, loss and optimizer. Initialize weights. For each mini-batch, perform a forward pass, compute loss, backpropagate gradients and update parameters. Repeat across epochs, monitor validation loss, tune with validation data and optionally use early stopping. Finally evaluate once on held-out test data.

Revision answer: Forward pass -> loss -> backpropagation -> update; repeat and validate.

3(c)(ii) | Derive the network output

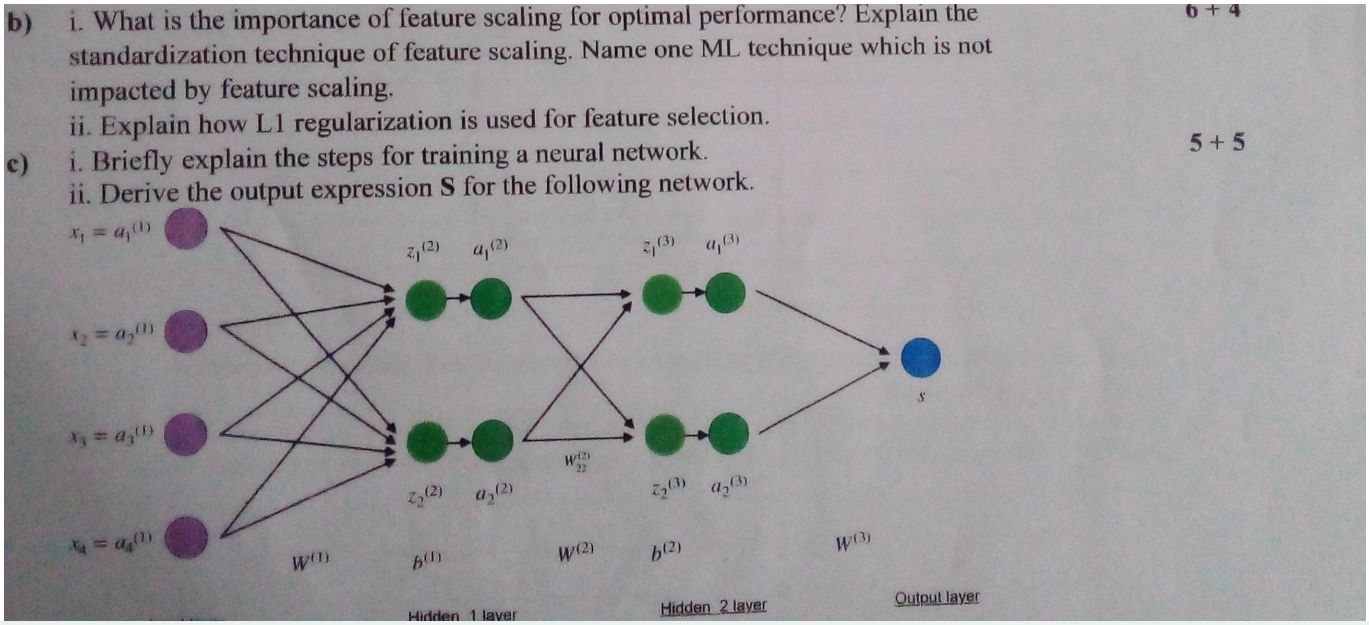
There are four input features, two neurons in hidden layer 1 and two in hidden layer 2. The small green z and a circles show preactivation and postactivation of the same neuron; they are not separate fully connected layers. Use column-vector convention, with w^(l)_{ji} connecting input i to output j.

z1^(2)=sum(i=1..4) w^(1)_{1i} x_i+b1^(1); a1^(2)=g1(z1^(2)). z2^(2)=sum(i=1..4) w^(1)_{2i} x_i+b2^(1); a2^(2)=g1(z2^(2)).

z1^(3)=w^(2)_{11}a1^(2)+w^(2)_{12}a2^(2)+b1^(2); a1^(3)=g2(z1^(3)).
z2^(3)=w^(2)_{21}a1^(2)+w^(2)_{22}a2^(2)+b2^(2); a2^(3)=g2(z2^(3)).

The diagram's final node is s with weights W^(3), and no output bias or activation is specified. Thus the direct weighted-sum interpretation is s=w^(3)_{11}a1^(3)+w^(3)_{12}a2^(3). If your lecture defines an output activation, write y=g_out(s); if it defines an output bias, add b_out explicitly.

In vector form without an extra output bias: s=W^(3) g2(W^(2) g1(W^(1)x+b^(1))+b^(2)). Shapes: W^(1):2×4; b^(1):2×1; W^(2):2×2; b^(2):2×1; W^(3):1×2. Weights/activations are unspecified, so this is a symbolic result.



Revision answer: s=W3·g2(W2·g1(W1·x+b1)+b2), with output activation/bias only if explicitly assumed.

4(a) | Main limitation of ensemble learning

Combining several models usually increases training cost, memory use and prediction latency, and often reduces interpretability. Improvement also depends on useful diversity: highly correlated weak models may add little benefit.

Revision answer: Higher computational cost and lower interpretability are common trade-offs.

4(b) | Nominal and ordinal features

Nominal values name categories without an inherent order: blood group A/B/AB/O or color red/green/blue. Ordinal values have a meaningful order but not necessarily equal spacing: satisfaction low/medium/high. Do not invent a numerical distance between nominal categories.

Revision answer: Nominal: blood group; ordinal: low/medium/high satisfaction.

4(c) | Batch gradient descent versus SGD

Batch gradient descent computes one gradient using the entire training set before updating weights: $\theta \leftarrow \theta - \eta \nabla J_{all}(\theta)$. It gives a stable full-data direction but each update may be expensive. Stochastic gradient descent uses one sampled observation per update, giving cheaper but noisy steps. Mini-batch gradient descent uses a small group and is the usual practical compromise. Libraries sometimes use "SGD" broadly for mini-batch training.

Revision answer: Batch: all examples per update; strict SGD: one example per update.

4(d) | Why differentiable activations?

Gradient-based learning uses derivatives through the chain rule to determine how changing weights changes loss. Activations should therefore be differentiable almost everywhere or have usable subgradients. ReLU is not differentiable at zero, but implementations choose a subgradient there, so it remains trainable.

Revision answer: Backpropagation needs derivatives or usable subgradients.

4(e) | Why k-NN is a lazy learner

k-NN stores the training examples and defers most modeling work until a query arrives, hence lazy learning. Resolve voting ties using a declared rule: distance-weighted voting, the class of the nearest tied neighbor, or a deterministic class order. Odd k reduces binary-class vote ties but does not eliminate multiclass ties or equal-distance ambiguities.

Revision answer: It postpones work until prediction; use an explicit tie-breaking rule.

4(f) | What are outliers?

Observations unusually far from the typical pattern. They may reflect measurement errors, rare valid events or a different data-generating process. Investigate before deleting them; automatic removal can discard important cases.

Revision answer: Outliers are atypical observations, not necessarily errors.

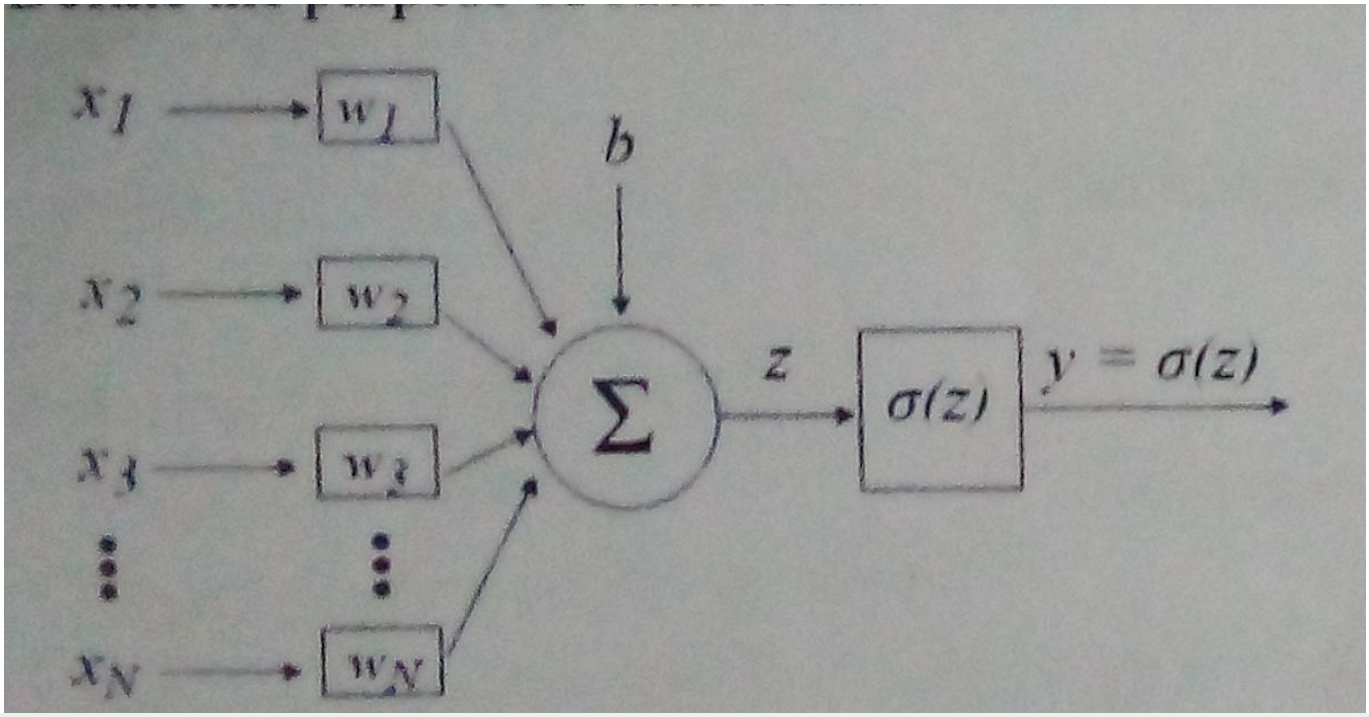
4(g) | Metrics for imbalanced classification

Inspect class-wise precision and recall, F1 or F-beta when recall/precision emphasis matters, and precision-recall curves/average precision for rare-positive ranking tasks. Balanced accuracy averages recalls across classes. Report the confusion matrix and threshold, and choose according to false-positive/false-negative costs. Ordinary accuracy alone can conceal failure on the minority class.

Revision answer: Use class-wise precision/recall, F1, PR analysis and balanced accuracy as appropriate.

5(a) | Variables of an artificial neuron

x1…xN are input features. w1…wN are learned weights controlling contribution. b is the learned bias shifting the decision threshold. Σ combines inputs: z=sum(w_i x_i)+b. σ is an activation function, which may be sigmoid, ReLU or another specified function. Output y=σ(z). For sigmoid, σ(z)=1/(1+e^(−z)); do not assume every σ label means sigmoid unless the lecture defines it that way.

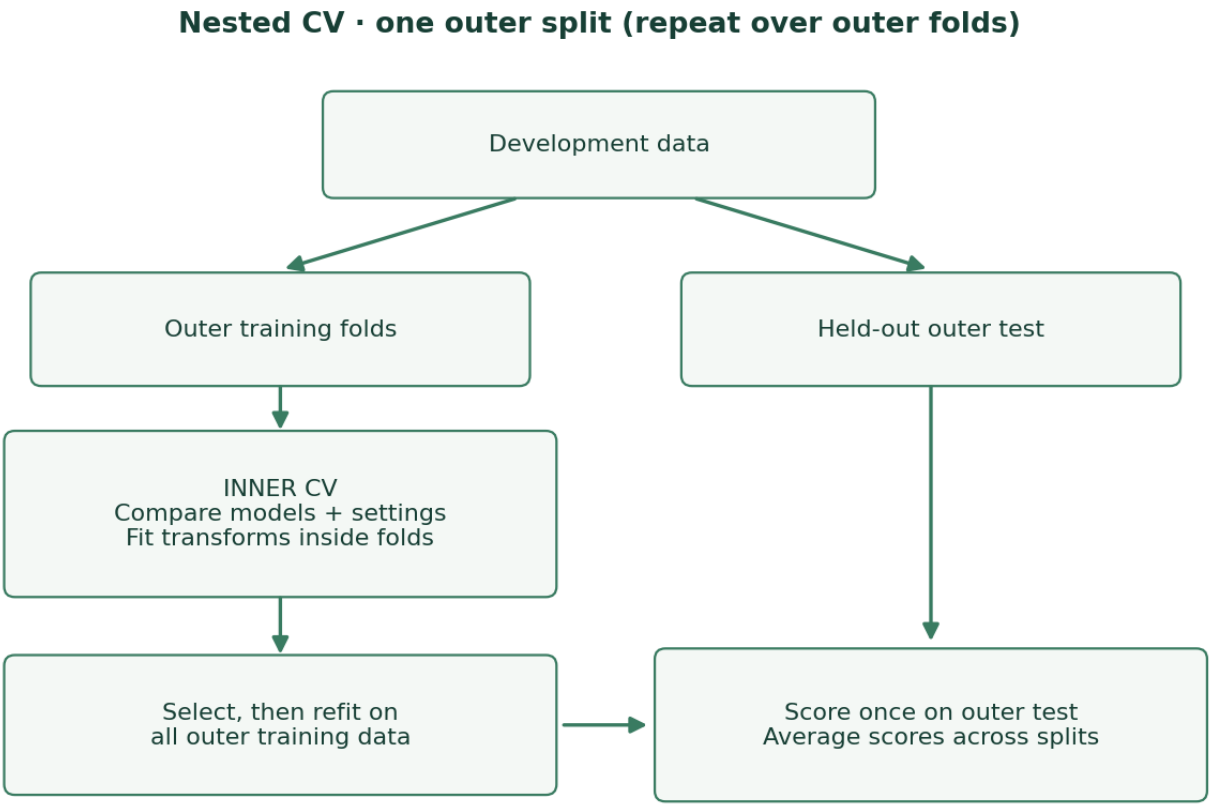


Revision answer: Inputs x, weights w, bias b, weighted sum z and activated output y.

5(b) | Nested k-fold CV: selection and tuning

The outer loop estimates generalization of the whole selection procedure. Hold one outer fold out as a test set. Using only the remaining outer training data, run an inner CV search over candidate model families and hyperparameters. Fit preprocessing inside each inner training fold too.

Choose the best inner-validation configuration, refit it on all outer training data and score it once on the untouched outer test fold. Repeat for every outer fold and summarize outer scores. Do not tune on those outer test scores. After evaluating the procedure, run the same inner selection on all development data and refit a final model; use an independent final test only if reserved. The diagram illustrates one outer split and its inner search.



Reading: scikit-learn nested CV example - https://scikit-learn.org/stable/auto_examples/model_selection/plot_nested_cross_validation_iris.html

Revision answer: Inner CV selects models/settings; outer CV evaluates the complete selection procedure.

5(c) | One-hot encoding a nominal feature

For color categories [red,green,blue], create binary columns [is_red,is_green,is_blue]. Red becomes [1,0,0], green [0,1,0], blue [0,0,1]. This avoids falsely treating category codes as ordered numbers. Fit the category vocabulary using training data and declare how unknown values are handled.

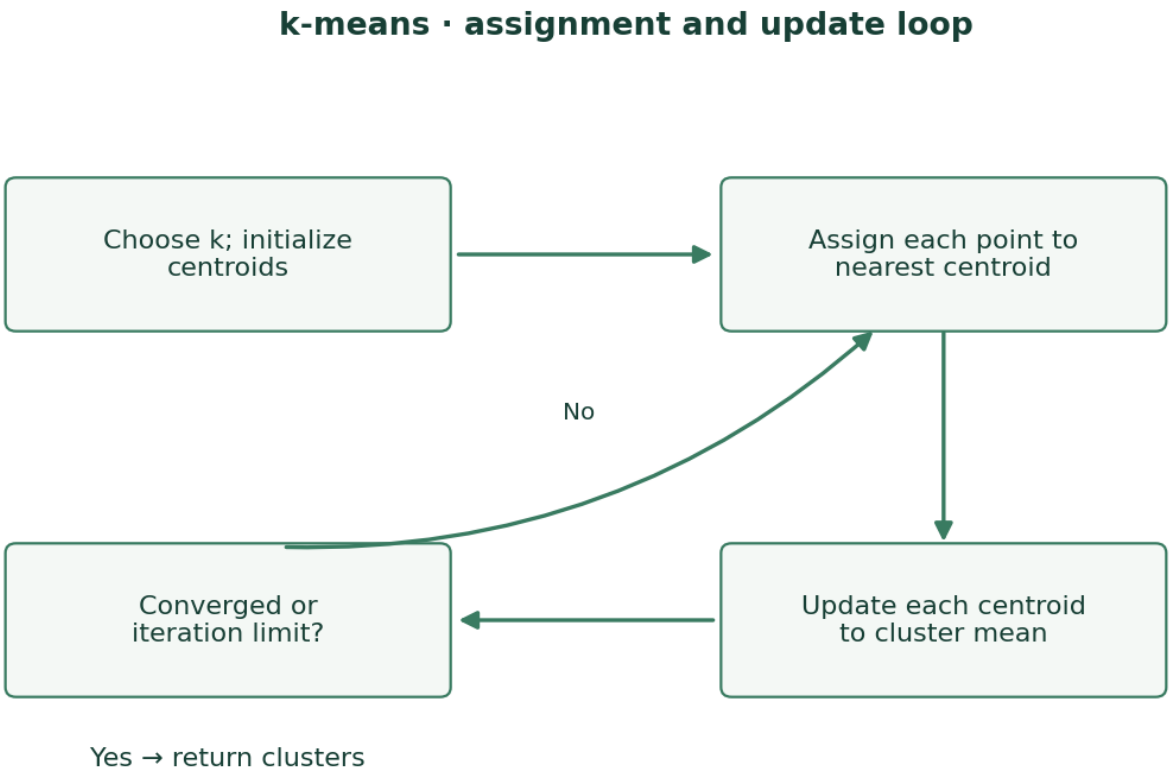
Main disadvantage: high cardinality can create many sparse columns, increasing dimension and processing/storage demands. For an unregularized linear model with an intercept, all category columns are also linearly dependent, so one reference category may be dropped.

Revision answer: Replace each category with a binary indicator; many categories can create high dimension.

5(d) | k-means algorithm and flowchart

Choose k and initialize k centroids, preferably with a suitable seeding method. Assign each observation to its nearest centroid under the selected distance. Recompute each centroid as the mean of its assigned points. Repeat assignment and update until assignments stabilize, centroid movement is sufficiently small or a maximum iteration count is reached.

The objective is within-cluster sum of squared Euclidean distances. Initialization can affect the local solution; use multiple starts. Scale features where appropriate and define how empty clusters are reinitialized. The diagram shows the iterative assignment/update loop.



Revision answer: Initialize → assign nearest cluster → update means → repeat until convergence.

5(e) | Max-pooling in a CNN

Max-pooling takes the maximum within each local window, typically reducing spatial dimensions and computation while retaining strong activations. It provides some tolerance to small local shifts, but loses spatial detail and is not universal invariance.

Example with 2×2 windows and stride 2: input rows [1,3,2,0]; [4,2,5,1]; [0,1,2,3]; [2,0,1,6]. Maxima are 4 for the top-left window, 5 top-right, 2 bottom-left and 6 bottom-right. Output rows are [4,5]; [2,6]. Pool each channel separately; ordinary max-pooling has no learned weights.

Revision answer: 2×2 stride-2 pooling turns the example into [[4,5],[2,6]].

5(f) | Why k-NN is nonparametric and how ties are resolved

k-NN does not fit a fixed finite-dimensional functional form such as one straight line. It retains examples, and its effective decision complexity can grow with the training set; hence nonparametric. It still has hyperparameters such as k and distance metric.

If class votes tie, use a declared deterministic rule, such as distance-weighted voting, the class of the closest tied neighbor or a fixed class order. Odd k reduces binary-class vote ties but does not remove all multiclass or equal-distance ties.

Revision answer: No fixed fitted function form; declare a distance-based or deterministic tie rule.

6(a)(i) | Entropy and first split: X, Y or Z?

Rows are (X,Y,Z,C): (1,1,1,I), (1,1,0,I), (0,0,1,II), (1,0,0,II). Parent has two I and two II, so H(parent)=1 bit.

Split X: X=0 has one II (H=0); X=1 has two I and one II, $H=-(2/3)\log_2(2/3)-(1/3)\log_2(1/3)=0.918296$. Weighted $H=(1/4)0+(3/4)0.918296=0.688722$; $IG(X)=0.311278$.

Split Y: Y=1 contains two I; Y=0 contains two II. Both are pure, weighted H=0; IG(Y)=1.

Split Z: Z=1 has one I and one II, and Z=0 has one I and one II. Both have H=1; weighted H=1; IG(Z)=0.

Choose Y: it gives the largest information gain and perfectly separates these four training rows.

Revision answer: IG(X)=0.311278, IG(Y)=1, IG(Z)=0; split first on Y.

6(a)(ii) | Bagging ensemble classifier

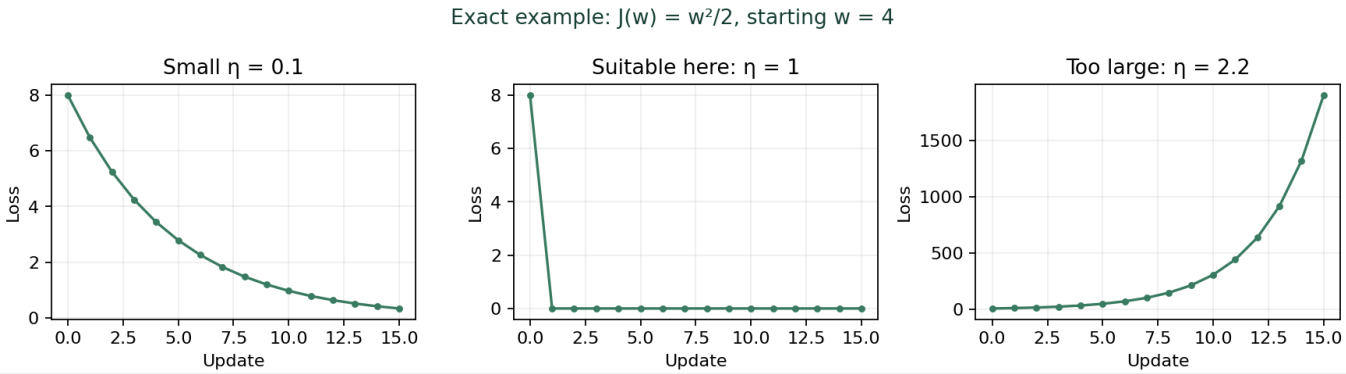
Create multiple bootstrap samples of the training set. Fit a separate base learner to each sample, usually independently. Combine classification outputs by majority/plurality voting or probability averaging; regression uses averaging. The diversity reduces variance when base learners are unstable and not perfectly correlated. Random forests add random feature selection to tree bagging. Out-of-bag cases can support internal evaluation; a held-out test remains separate.

Revision answer: Bootstrap training sets → fit independent learners → aggregate their predictions.

6(b)(i) | Learning rate: too small, suitable, too large

Gradient descent updates $\theta_{new}=\theta_{old}-\eta\nabla J(\theta_{old})$; η is the learning rate controlling step size. A very small η takes tiny steps and converges slowly; a suitable η reduces loss efficiently; an excessive η overshoots and can oscillate or diverge.

For $J(w)=w^2/2$, gradient=w and $w_{new}=(1-\eta)w$. With w0=4: η=0.1 gives 4,3.6,3.24,...; η=1 gives 4,0,0,...; η=2.2 gives 4,−4.8,5.76,... and diverges. These exact examples illustrate the principle, not universal learning-rate recommendations. The figure shows loss across iterations for the three cases.

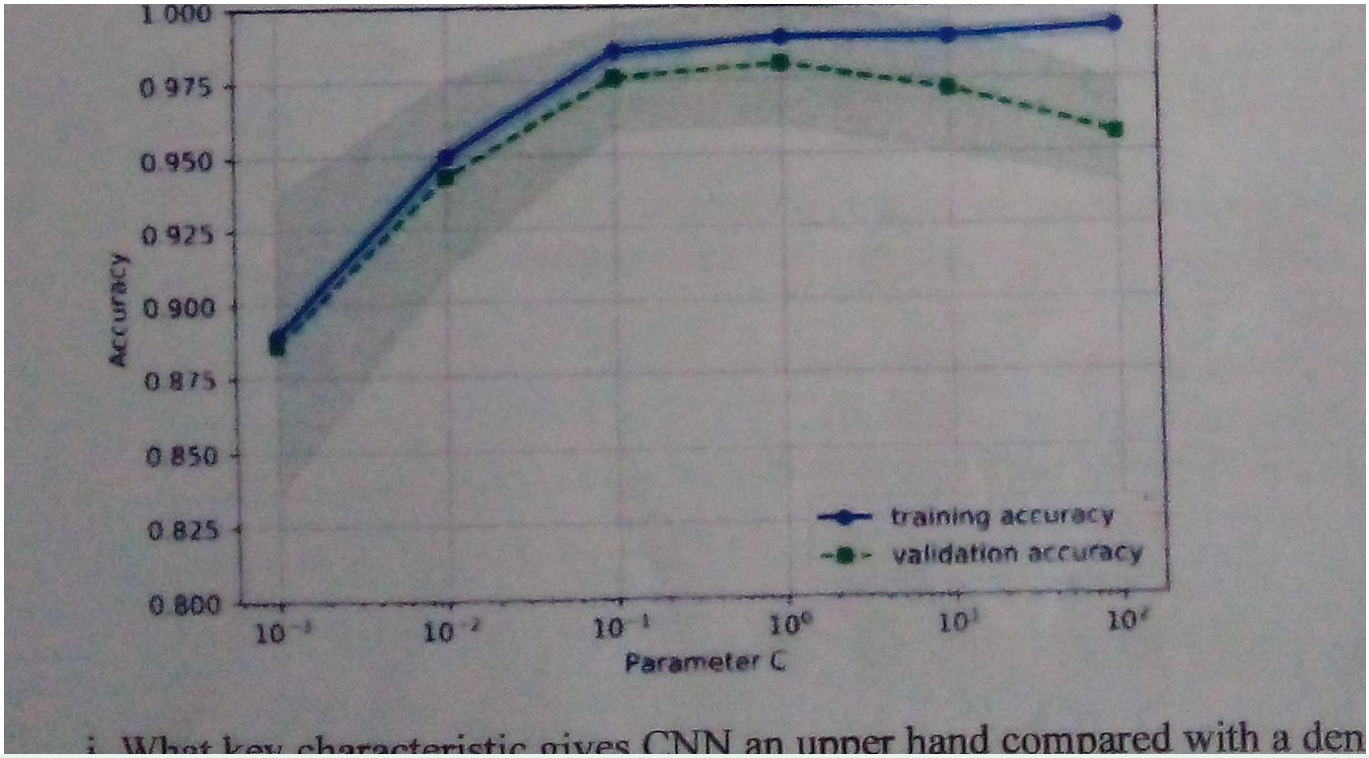


Revision answer: Learning rate controls step size; too small is slow and too large can diverge.

6(b)(ii) | Interpret training/validation accuracy versus C

In the common logistic-regression convention in this plot, C is inverse regularization strength. At very small C, strong regularization limits the model, so both training and validation accuracy are relatively low: underfitting. As C increases, both improve. Validation accuracy reaches its best plotted region around C=1; choose settings using validation or CV, not training accuracy.

At larger C, training accuracy stays high or rises, while validation accuracy falls and the gap widens: overfitting with weaker regularization. The graph is qualitative; do not claim exact accuracy decimals from the photograph. C≈1 is the best visible candidate among those plotted, not a universal optimum.



Revision answer: Small C underfits; around C=1 validates best; large C weakens regularization and overfits.

6(c)(i) | Why CNNs suit image classification

Local receptive fields exploit nearby pixel structure; shared convolution weights detect the same feature across locations using far fewer parameters than a fully connected image layer. Stacked layers build spatial feature hierarchies. Convolution is translation-equivariant away from boundary effects, while pooling/aggregation can provide some shift tolerance; a CNN is not automatically invariant to every shift or distortion.

Revision answer: Local connectivity and shared filters exploit spatial structure efficiently.

6(c)(ii) | 3×3 convolution on a 5×5 input, stride 2

Assume no padding (valid convolution), since none is specified. Output height=width=floor((5−3)/2)+1=2, so one filter extracts 2×2=4 patches. Zero-based top-left coordinates are (0,0), (0,2), (2,0), (2,2). Each patch is 3×3×1; multiply it elementwise by the shared kernel and sum, adding a bias if defined.

No input or kernel values are given, so numerical feature values cannot be computed. With "same" padding, output size would instead be ceiling(5/2)=3 per axis, or 9 padded patches. The valid assumption is therefore essential.

Revision answer: With valid/no padding: 2×2 output, 4 patches; same padding would give 9.

Reference reading

Christopher M. Bishop, Pattern Recognition and Machine Learning.

Aurelien Geron, Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow.

scikit-learn User Guide: https://scikit-learn.org/stable/user_guide.html

Keras model training API: https://keras.io/api/models/model_training_apis/

Nested CV example: https://scikit-learn.org/stable/auto_examples/model_selection/plot_nested_cross_validation_iris.html

These are supporting reading references, not claims that this paper or its answer key appears in those books. Course-specific terminology and source ambiguities are noted beside the affected answers.

Shahjalal University of Science and Technology

Department of Computer Science and Engineering

4th Year 1st Semester Final Examination, June 2024 (Session: 2019-20)

Course Code: CSE 475 Course Title: Machine Learning

Time: 3.0 hrs **Credits: 3.0** **Total Marks: 100**

Group A

[Answer all questions]

1. Answer any FIVE. **5x2 = 10**

a) What is plurality voting?

b) Name two loss functions.

c) What is hyperparameter? Give examples.

d) What is meant by a bootstrap sample?

e) Give two examples of reinforcement learning.

f) In which case, the entropy is zero in decision tree algorithm?

g) What is meant by high bias?

2. Answer any FOUR. **4x5 = 20**

a) Explain the meaning of the following piece of Python code.

```
model.compile(optimizer="rmsprop",
              loss="binary_crossentropy",
              metrics=["accuracy"])
model.fit(x_train, y_train, epochs=4, batch_size=512)
```

b) What is the main limitation of perceptron algorithm for classification? How is it resolved with the logistic regression method? Explain.

c) Why is dimensionality reduction required in ML algorithms? Discuss the PCA technique to achieve data compression.

d) Define the term TF-IDF. How does TF-IDF assess the word relevancy in encoding?

e) You are asked to determine the most discriminative feature in a dataset. How will you do it?

f) How is the nonlinearly separable case dealt using slack variables in SVM? Explain.

2x10 = 20

f) How is the nonlinearly separable case dealt using slack variables in SVM? Explain.

3. Answer any TWO. **2x10 = 20**

a) Suppose you have written a classifier for email spam filtering. You have tested your classifier with some data and got the following predictions. Compute the confusion matrix from the output and calculate **Accuracy**, **Sensitivity**, **Precision, and f1 score** from that matrix **3 + 7**

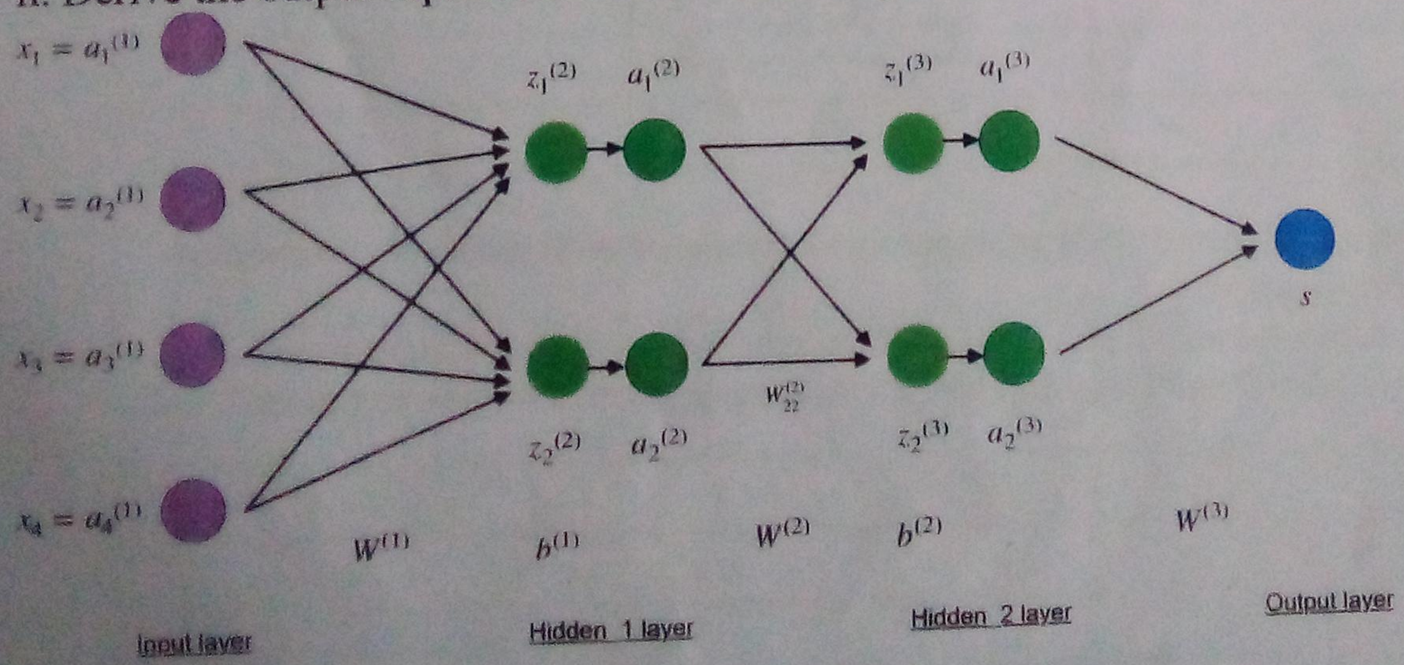
SL.	Target	Prediction
1	Spam	Not Spam
2	Spam	Spam
3	Not Spam	Not Spam
4	Spam	Spam
5	Not Spam	Spam
6	Not Spam	Not Spam
7	Not Spam	Not Spam
8	Spam	Spam
9	Spam	Not Spam
10	Not Spam	Not Spam

b) i. What is the importance of feature scaling for optimal performance? Explain the standardization technique of feature scaling. Name one ML technique which is not impacted by feature scaling. **6 + 4**

ii. Explain how L1 regularization is used for feature selection.

c) i. Briefly explain the steps for training a neural network. **5 + 5**

ii. Derive the output expression **S** for the following network.



Group B

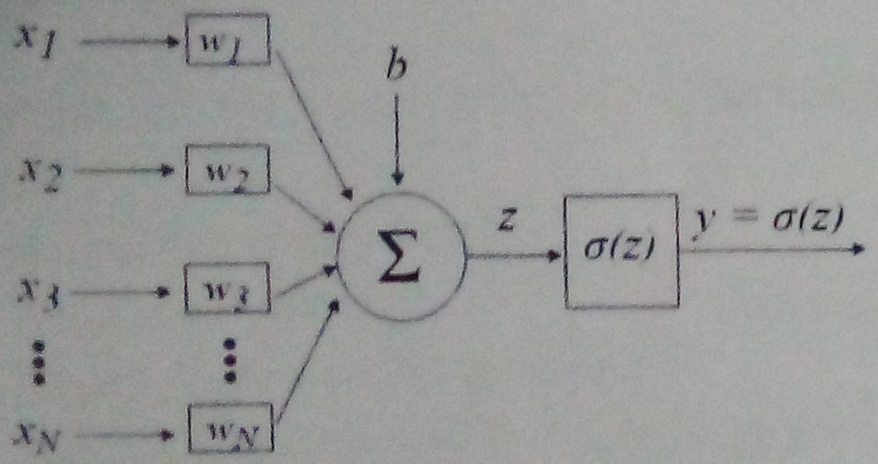
[Answer all questions]

4. Answer any FIVE. **5x2 = 10**

a) What is the main limitation of ensemble learning?
b) Give example of a nominal and an ordinal feature.
c) How does batch gradient descent differ from SGD?
d) Why should the activation function in ANN be differentiable?
e) Why is KNN called a lazy learner?
f) What are outliers?
g) Which evaluation metrics are good for imbalanced data?

5. Answer any FOUR. **4x5 = 20**

a) Define the purpose of each of the variables in the following figure.



b) Explain with a figure how a model selection is achieved together with tuning hyperparameters using nested k-fold cross-validation.
c) Show with an example how one-hot encoding is performed on a nominal feature? What is the main disadvantage of this approach?
d) How does K-Means clustering works? Draw the flowchart of K-Means clustering.
e) What is the purpose of max-pooling in CNN? Illustrate the process with an example.
f) Why KNN is termed as a non-parametric method? What happens when the majority voting in KNN results in a tie?

6. Answer any TWO. **2x10 = 20**

a) i. For the following dataset with 3 features (X, Y, Z) and 2 classes (I, II), Which attribute to split first according to entropy in decision tree algorithm? **6 + 4**



b) Explain with a figure how a model selection is achieved together with tuning hyperparameters using nested k-fold cross-validation.

c) Show with an example how one-hot encoding is performed on a nominal feature? What is the main disadvantage of this approach?

d) How does K-Means clustering works? Draw the flowchart of K-Means clustering.

e) What is the purpose of max-pooling in CNN? Illustrate the process with an example.

f) Why KNN is termed as a non-parametric method? What happens when the majority voting in KNN results in a tie?

2x10 = 20

6. Answer any TWO.

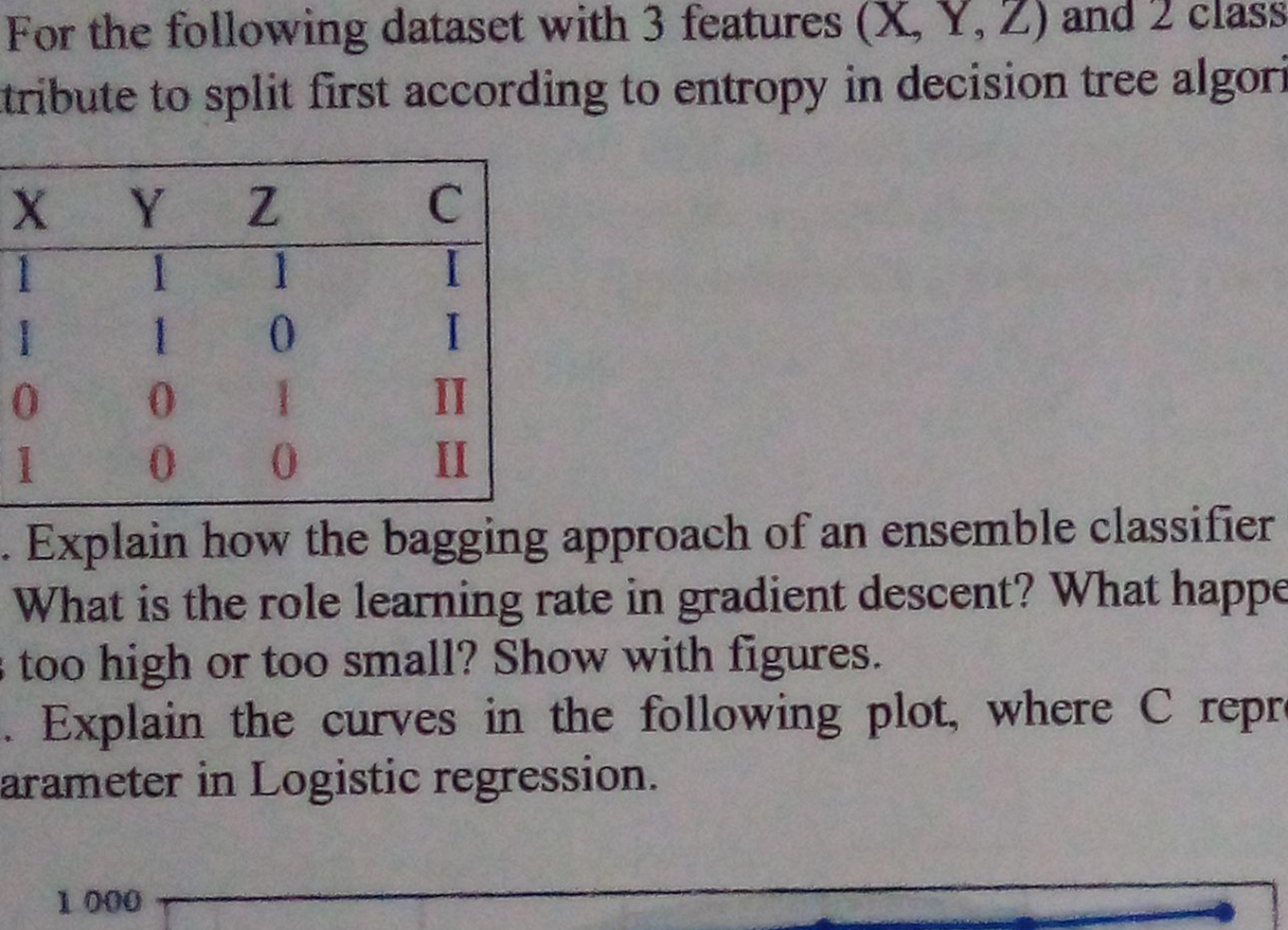
a) i. For the following dataset with 3 features (X, Y, Z) and 2 classes (I, II), Which attribute to split first according to entropy in decision tree algorithm? **6 + 4**

X	Y	Z	C
1	1	1	I
1	1	0	I
0	0	1	II
1	0	0	II

ii. Explain how the bagging approach of an ensemble classifier works?

b) i. What is the role learning rate in gradient descent? What happens when the learning rate is too high or too small? Show with figures. **5 + 5**

ii. Explain the curves in the following plot, where C represents the regularization parameter in Logistic regression.



c) i. What key characteristic gives CNN an upper hand compared with a densely connected layer in image classification? **3 + 7**

ii. Describe with how a kernel of size 3×3 is convolved with an input feature map of size 5×5 with stride 2×2 and input depth = 1. How many patches will be extracted?